

Documentação do Projeto — G&P Agência Espacial

Última atualização: 11 de janeiro de 2026

Visão Geral

G&P Agência Espacial é um pequeno site estático em React que apresenta seções institucionais: cabeçalho, destaque de propulsores, inscrição em newsletter, equipe, carrossel de notícias e rodapé. O projeto usa Vite + React e Sass para estilos.

Objetivo desta documentação: descrever a arquitetura, estrutura dos arquivos, componentes, hooks, estilos, scripts de execução e instruções para contribuir e gerar a documentação em PDF.

Estrutura do projeto

Raiz do projeto (principais arquivos e pastas):

- `index.html` — arquivo HTML principal
- `package.json` — scripts e dependências
- `vite.config.js` — configuração do Vite
- `src/` — código-fonte React
 - `App.jsx` — componente raiz
 - `index.jsx` — entrada da aplicação
 - `components/` — componentes reutilizáveis
 - `Header.jsx`, `Footer.jsx`, `Logo.jsx`, `Alert.jsx`, `Newsletter.jsx`, `Noticias.jsx`, `Equipe.jsx`, `Propulsores.jsx`
 - `hooks/` — hooks customizados
 - `useNoticiasCarousel.jsx`, `useSendNewsletter.jsx`
 - `styles/` — arquivos Sass
 - `global.sass`, além de `Header.sass`, `Footer.sass`, `Newsletter.sass`, etc.
 - `assets/Imagens/` — imagens usadas pela interface
- `legacy/` — versão antiga / exemplo estático
- `public/` — arquivos públicos servidos diretamente

Dependências e scripts

Dependências principais (em `package.json`):

- `react`, `react-dom`
- `sass` (para compilar arquivos `.sass`)

Scripts relevantes (via npm):

- `npm run dev` — inicia servidor de desenvolvimento (Vite)
- `npm run build` — gera build de produção
- `npm run preview` — serve o build gerado
- `npm run lint` — roda ESLint (configuração do projeto)

Componentes (resumo)

- `App.jsx` :

- Componente raiz que importa todos os componentes e estilos globais.
- Define `window.alerta` usando uma ref para o componente `Alert`, permitindo exibir mensagens globais de alerta.
- `Header.jsx` :
 - Exibe logo e menus. Links e botões chamam `window.alerta()` (funcionalidades ainda em construção).
- `Propulsores.jsx` :
 - Seção destacada com texto sobre fabricação de propulsores.
- `Newsletter.jsx` :
 - Formulário simples para capturar e-mail.
 - Usa o hook `useSendNewsletter` para validação e envio (apenas comportamento de UI, sem backend integrado).
- `Equipe.jsx` :
 - Seção com descrição da equipe e imagem ilustrativa.
- `Noticias.jsx` :
 - Carrossel de notícias com imagens, títulos e descrições.
 - Controles: anterior/próxima e indicadores (pontos).
 - Chamadas a `window.alerta()` para links "Sobre mais..." que não estão implementados.
- `Footer.jsx` :
 - Logo e ícones de redes sociais (alguns com links externos).
- `Alert.jsx` :
 - Componente controlado via `ref` que exibe alertas cobrindo a tela.
 - Expondo métodos `showAlert(msg)` e `hideAlert()` via `useImperativeHandle`.

Hooks customizados

- `useNoticiasCarousel.jsx` :
 - Lida com lógica do carrossel: centralização de slides, controles, indicadores e autoplay (10s).
 - Usa seleção direta do DOM (`querySelector`) para manipular elementos e adicionar/remover listeners.
- `useSendNewsletter.jsx` :
 - Gerencia referência do formulário e validação de e-mail (regex).
 - Mostra mensagens via `window.alerta()`; não faz chamadas a APIs externas.

Estilos

- Projeto utiliza Sass (`.sass`) com arquivo global `global.sass` e arquivos específicos por componente.
- Há regras básicas de reset e componentes estilizados com classes e IDs (por exemplo, `#login`, `#cadastrar`, `#enviar`).

Assets

- Imagens localizadas em `src/assets/Imagens/` usadas por `Noticias` , `Footer` , `Equipe` , etc.

Acessibilidade e notas de usabilidade

- Alguns links e botões usam `onClick` sem `role` / `tabIndex` explícitos; considere melhorar acessibilidade adicionando atributos ARIA e garantindo foco por teclado.
- O componente `Alert` bloqueia a rolagem do `body` quando ativo — comportamento intencional para modal.

Responsividade

- A tarefa original indica que o site deve ser responsivo (celular, tablet, notebook, FullHD). Os Sass devem conter media queries (revise `*.sass` para ajustes finos de layout).

Como rodar o projeto localmente

1. Instale dependências:

```
npm install
```

2. Ambiente de desenvolvimento (inicia Vite):

```
npm run dev
```

3. Build de produção:

```
npm run build  
npm run preview
```

4. Lint (se configurado):

```
npm run lint
```

Gerar esta documentação como PDF (automatizado)

Este repositório agora contém um arquivo Markdown com esta documentação em `docs/PROJECT_DOCUMENTATION_PT.md` . Para gerar um PDF localmente (será utilizado o pacote `md-to-pdf`):

```
npm i -D md-to-pdf  
npx md-to-pdf docs/PROJECT_DOCUMENTATION_PT.md -o docs/PROJECT_DOCUMENTATION_PT.pdf
```

Observação: a geração de PDF requer Node.js + npm e poderá baixar binários (se o `md-to-pdf` usar puppeteer/Chromium).

Sugestões de melhorias

- Separar lógica do DOM do `useNoticiasCarousel` para torná-lo baseado em refs/estado React, melhorando testabilidade.
- Adicionar testes unitários (Jest/React Testing Library).
- Implementar integração real para newsletter (API/email service).
- Melhorar acessibilidade (teclado, ARIA, labels para formulários).
- Adicionar CI (GitHub Actions) para lint e build automático.

Contribuição

- Fork + branch => PR com descrição clara das mudanças.
- Siga convenções de código e execute `npm run lint` antes de abrir PR.

Contato

- Arquivo `Footer.jsx` contém links para redes sociais usadas pelo projeto. Para dúvidas, abra uma Issue no repositório.

Arquivo gerado automaticamente por assistente de documentação.